

Diario de trabajo:

La resolución de estos problemas se ha llevado a cabo en varios días, en primer lugar realicé una lectura de la práctica 2a y posteriormente pasé a la realización de los ejercicios: viernes 12 de octubre y martes 21 de octubre.

Códigos fuente:

Código 1

```
.section .data
lista:      .int 0x10000000, 0x10000000, 0x10000000, 0x10000000, 0x10000000,
0x10000000, 0x10000000, 0x10000000, 0x10000000, 0x10000000, 0x10000000, 0x10000000,
0x10000000, 0x10000000, 0x10000000, 0x10000000
#lista: .int 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
#lista: .int 0x0ffffff, 0x0ffffff, 0x0ffffff, 0x0ffffff, 0x0ffffff, 0x0ffffff, 0x0ffffff, 0x0ffffff,
0x0ffffff, 0x0ffffff, 0x0ffffff, 0x0ffffff, 0x0ffffff, 0x0ffffff, 0x0ffffff, 0x0ffffff
#lista: .int 0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff,
0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff
longlista:  .int  (-lista)/4
resultado:  .quad  0
formato:    .asciz  "suma = %lu = 0x%lx hex\n"
```

3) gcc media.s -o media -no-pie -nostartfiles 6544 B

```
.section .text
```

```
_start: .global _start
```

```
main: .global main
      call trabajar  # subrutina de usuario
      call imprim_C # printf() de libC
      call acabar_C # exit() de libC
      ret
```

```
trabajar:
      mov  $lista, %rbx
      mov  longlista, %ecx
      call suma      # == suma(&lista, longlista);
      mov  %eax, resultado
      mov  %edx, resultado+4
      ret
```

```
suma:
      mov  $0, %eax
      mov  $0, %rsi
      mov  $0, %edx
```

```
bucle:
      add  (%rbx,%rsi,4), %eax
      jnc nocarry
```

```

        add $1, %edx
nocarry:
        inc %rsi
        cmp %rsi,%rcx
        jne bucle
        ret

imprim_C:                # requiere libC
        mov  $formato, %rdi
        mov  resultado,%rsi
        mov  resultado,%rdx
        mov  $0,%eax      # varargin sin xmm
        call printf        # == printf(formato, res, res);
        ret

acabar_C:                # requiere libC
        mov  resultado, %edi
        call _exit        # == exit(resultado)
        ret

```

Código 2

```

.section .data
    .macro linea
        #.int 1,1,1,1
        #.int 0x0fffffff,0x0fffffff,0x0fffffff,0x0fffffff
        #.int 0x10000000,0x10000000,0x10000000,0x10000000
        .int 0xffffffff,0xffffffff,0xffffffff,0xffffffff
    .endm
lista: .irpc i,1234
    linea
    .endr
longlista:    .int  (-lista)/4
resultado:    .quad  0
formato:      .asciz  "suma = %lu = 0x%lx hex\n"

# 3) gcc media2.s -o media2 -no-pie -nostartfiles          6544 B

.section .text

_start: .global _start

main: .global main
    call trabajar    # subrutina de usuario
    call imprim_C    # printf() de libC
    call acabar_C    # exit() de libC
    ret

trabajar:
    mov  $lista, %rbx
    mov  longlista, %ecx
    call suma        # == suma(&lista, longlista);

```

```

    mov %eax, resultado
    mov %edx, resultado+4
    ret

```

suma:

```

    mov $0, %eax
    mov $0, %rsi
    mov $0, %edx

```

bucle:

```

    add (%rbx,%rsi,4), %eax
    adc $0, %edx
    inc %rsi
    cmp %rsi,%rcx
    jne bucle
    ret

```

imprim_C: # requiere libC

```

    mov $formato, %rdi
    mov resultado,%rsi
    mov resultado,%rdx
    mov $0,%eax # varargin sin xmm
    call printf # == printf(formato, res, res);
    ret

```

acabar_C: # requiere libC

```

    mov resultado, %edi
    call _exit # == exit(resultado)
    ret

```

Código 3

```

.section .data
.macro linea
    .int 1,1,1,1
    #.int 0x0fffffff,0x0fffffff,0x0fffffff,0x0fffffff
    #.int 0x10000000,0x10000000,0x10000000,0x10000000
    #.int -1, -1, -1, -1
    #.int 0xffffffff,0xffffffff,0xffffffff,0xffffffff
    #.int -1000000000, -1000000000, -1000000000, -1000000000
    #.int -2000000000, -2000000000, -2000000000, -2000000000
    #.int -3000000000, -3000000000, -3000000000, -3000000000
    #.int -20000000000, -20000000000, -20000000000, -20000000000
.endm
lista: .irpc i,1234
    linea
.endr
longlista: .int (.-lista)/4
resultado: .quad 0
formato: .asciz "suma = %ld = 0x%lx hex\n"

```

3) gcc media3.s -o media3 -no-pie -nostartfiles

6544 B

```
.section .text
```

```
_start: .global _start
```

```
main: .global main
```

```
    call trabajar    # subrutina de usuario
    call imprim_C    # printf() de libC
    call acabar_C    # exit() de libC
    ret
```

```
trabajar:
```

```
    mov    $lista, %rbx
    mov    longlista, %ecx
    call suma        # == suma(&lista, longlista);
    mov    %r8d, resultado
    mov    %r9d, resultado+4
    ret
```

```
suma:
```

```
    mov    $0, %eax
    mov    $0, %rsi
    mov    $0, %edx
    mov    $0, %r8d
    mov    $0, %r9d
```

```
bucle:
```

```
    mov    (%rbx,%rsi,4), %eax
    cld    #Convert Long to Double, sirve para extender el signo de eax a edx:eax
    add    %eax, %r8d
    adc    %edx, %r9d
    inc    %rsi
    cmp    %rsi,%rcx
    jne    bucle
    ret
```

```
imprim_C:                # requiere libC
```

```
    mov    $formato, %rdi
    mov    resultado,%rsi
    mov    resultado,%rdx
    mov     $0,%eax    # varargin sin xmm
    call    printf      # == printf(formato, res, res);
    ret
```

```
acabar_C:                # requiere libC
```

```
    mov    resultado, %edi
    call    _exit        # == exit(resultado)
    ret
```

Código 4

```
.section .data

#ifdef TEST
#define TEST 20
#endif
    # doble macro: lista definida por una linea0 y 3 lineas normales
    # mayoría ejemplos linea0 = linea => lista tiene 4 lineas normales
    .macro linea    # Media / Resto - Comentario

#ifdef TEST==1    //1 / 8
    .int 1, 2, 1, 2
#elif TEST==2    //-1 /-8
    .int -1,-2,-1,-2
#elif TEST==3    //
    .int 0x7fffffff,0x7fffffff,0x7fffffff,0x7fffffff
#elif TEST==4    //
    .int 0x80000000,0x80000000,0x80000000,0x80000000
#elif TEST==5    //
    .int 0xffffffff,0xffffffff,0xffffffff,0xffffffff
#elif TEST==6    //
    .int 2000000000,2000000000,2000000000,2000000000
#elif TEST==7    //
    .int 3000000000,3000000000,3000000000,3000000000
#elif TEST==8    //
    .int -2000000000,-2000000000,-2000000000,-2000000000
#elif TEST==9    //
    .int -3000000000,-3000000000,-3000000000,-3000000000
#elif TEST>=10 && TEST<=14
    .int 1, 1, 1, 1    # linea0 + 3lineas, casi todo a 1
#elif TEST>=15 && TEST<=19
    .int -1,-1,-1,-1    # linea0 + 3lineas, casi todo -1
#else
    .error "Definir TEST entre 1..19"
#endif
    .endm

    # mayoría ejemplos linea0 = linea => lista tiene 4 lineas normales

    .macro linea0    # Media / Resto - Comentario
#ifdef TEST>=1 && TEST<=9
    linea    # casi siempre 4 lineas iguales
#elif TEST==10    //1 / 0 - equiv toda lista 1
    .int 0, 2, 1, 1
#elif TEST==11    //
    .int 1, 2, 1, 1
#elif TEST==12    //
    .int 8, 2, 1, 1
#elif TEST==13    // //
    .int 15,2,1,1
#elif TEST==14    // // //
    .int 16,2,1,1
```

```

#elif TEST==15 // -1 / 0 - equiv TEST05
    .int 0,-2,-1,-1
#elif TEST==16 //
    .int -1,-2,-1,-1
#elif TEST==17 //
    .int -8,-2,-1,-1
#elif TEST==18 //
    .int -15,-2,-1,-1
#elif TEST==19 //? / ?
    .int -16,-2,-1,-1
#else
    .error "Definir TEST entre 1..19"
#endif
.endm
# mayoría ejemplos linea0 = linea => lista tiene 4 lineas normales
lista: linea0
    .irpc i,123
        linea
    .endr

longlista:    .int  (-lista)/4
media: .int  0
resto:  .int  0

formato: .ascii "media \t = %11d \t resto \t = %11d \n"
.asciz "\t = 0x %08x \t \t = 0x %08x\n"

# gcc media4.s -o media4 -no-pie -nostartfiles
# for i in $(seq 1 19); do gcc -x assembler-with-cpp -D TEST=$i -no-pie -nostartfiles media4.s -o
media4; printf "__TEST%02d__%35s\n" $i "" | tr " " "-"; ./media4 ; done

.section .text

_start: .global _start

main: .global main
    call trabajar    # subrutina de usuario
    call imprim_C# printf() de libC
    call acabar_C # exit() de libC
    ret

trabajar:
    mov  $lista, %rbx
    mov  longlista, %ecx
    call suma          # == suma(&lista, longlista);
    mov  %eax, media
    mov  %edx, resto
    ret

suma:
    mov  $0, %eax
    mov  $0, %rsi

```

```

        mov $0, %edx
mov $0, %r8d
mov $0, %r9d

bucle:
        mov (%rbx,%rsi,4), %eax
        cltd
        add %eax, %r8d
        adc %edx, %r9d
        inc %rsi
        cmp %rsi,%rcx
        jne bucle
        mov %r8d, %eax
        mov %r9d, %edx
        idiv %ecx
        ret

imprim_C:                                # requiere libC
        mov $formato, %rdi
        mov media,%rsi
        mov resto,%rdx
        mov media, %rcx
        mov resto, %r8
        mov     $0,%eax    # varargin sin xmm
        call printf        # == printf(formato, res, res);
        ret

acabar_C:                                # requiere libC
        mov media, %edi
        call _exit         # == exit(resultado)
        ret

```

Código 5

```

.section .data

#ifdef TEST
#define TEST 20
#endif
    # doble macro: lista definida por una linea0 y 3 lineas normales
    # mayoría ejemplos linea0 = linea => lista tiene 4 lineas normales
    .macro linea    # Media / Resto - Comentario

    #if TEST==1    //1 / 8
        .int 1, 2, 1, 2
    #elif TEST==2  //-1 /-8
        .int -1,-2,-1,-2
    #elif TEST==3  //
        .int 0x7fffffff,0x7fffffff,0x7fffffff,0x7fffffff
    #elif TEST==4  //
        .int 0x80000000,0x80000000,0x80000000,0x80000000
    #elif TEST==5  //

```

```

.int 0xffffffff,0xffffffff,0xffffffff,0xffffffff
#elif TEST==6 //
.int 2000000000,2000000000,2000000000,2000000000
#elif TEST==7 //
.int 3000000000,3000000000,3000000000,3000000000
#elif TEST==8 //
.int -2000000000,-2000000000,-2000000000,-2000000000
#elif TEST==9 //
.int -3000000000,-3000000000,-3000000000,-3000000000
#elif TEST>=10 && TEST<=14
.int 1, 1, 1, 1 # linea0 + 3lineas, casi todo a 1
#elif TEST>=15 && TEST<=19
.int -1,-1,-1,-1 # linea0 + 3lineas, casi todo -1
#else
.error "Definir TEST entre 1..19"
#endif
.endm

# mayoría ejemplos linea0 = linea => lista tiene 4 lineas normales

.macro linea0 # Media / Resto - Comentario
#if TEST>=1 && TEST<=9
.linea # casi siempre 4 lineas iguales
#elif TEST==10 //1 / 0 - equiv toda lista 1
.int 0, 2, 1, 1
#elif TEST==11 //
.int 1, 2, 1, 1
#elif TEST==12 //
.int 8, 2, 1, 1
#elif TEST==13 // //
.int 15,2,1,1
#elif TEST==14 // // //
.int 16,2,1,1
#elif TEST==15 // -1 / 0 - equiv TEST05
.int 0,-2,-1,-1
#elif TEST==16 //
.int -1,-2,-1,-1
#elif TEST==17 //
.int -8,-2,-1,-1
#elif TEST==18 //
.int -15,-2,-1,-1
#elif TEST==19 //? / ?
.int -16,-2,-1,-1
#else
.error "Definir TEST entre 1..19"
#endif
.endm
# mayoría ejemplos linea0 = linea => lista tiene 4 lineas normales
lista: linea0
.irpc i,123
.linea
.endr

```

```

longlista:    .int  (-lista)/4
media: .int  0
resto:  .int  0

formato: .ascii "media \t = %11d \t resto \t = %11d \n"
.asciz "\t = 0x %08x \t \t = 0x %08x\n"

# gcc media4.s -o media4 -no-pie -nostartfiles
# for i in $(seq 1 19); do gcc -x assembler-with-cpp -D TEST=$i -no-pie -nostartfiles media4.s -o
media4; printf "__TEST%02d__%35s\n" $i "" | tr " " "-"; ./media4 ; done

.section .text

_start: .global _start

main: .global main
      call trabajar  # subrutina de usuario
      call imprim_C # printf() de libC
      call acabar_C # exit() de libC
      ret

trabajar:
      mov  $lista, %rbx
      mov  longlista, %ecx
      call suma      # == suma(&lista, longlista);
      mov  %eax, media
      mov  %edx, resto
      ret

suma:
      mov  $0, %eax
      mov  $0, %rsi
      mov  $0, %edx
      mov  $0, %r8d
      mov  $0, %r9d

bucle:
      mov  (%rbx,%rsi,4), %eax
      cltq  #Convert Long to Quad, sirve para extender el signo de eax a rdx:rax
      add  %rax, %r8
      inc  %rsi
      cmp  %rsi,%rcx
      jne  bucle
      mov  %r8, %rax
      cqto  #Convert Quad to Octo, sirve para extender el signo de rax a rdx:rax
      idiv %rcx
      ret

imprim_C:      # requiere libC
      mov  $formato, %rdi
      mov  media,%rsi

```

```

        mov  resto,%rdx
mov media, %rcx
mov resto, %r8
        mov  $0,%eax  # varargin sin xmm
        call printf    # == printf(formato, res, res);
        ret

acabar_C:                # requiere libC
        mov  media, %edi
        call _exit      # == exit(resultado)
        ret

```

Código 1

```

alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media.s
-o media -no-pie -nostartfiles;./media
suma = 4294967296 = 0x100000000 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media.s
-o media -no-pie -nostartfiles;./media
suma = 16 = 0x10 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media.s
-o media -no-pie -nostartfiles;./media
suma = 4294967280 = 0xffffffff0 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media.s
-o media -no-pie -nostartfiles;./media
suma = 68719476720 = 0xfffffffff0 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ █

```

Código 2

```

alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media2.s
-o media2 -no-pie -nostartfiles;./media2
suma = 16 = 0x10 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media2.s
-o media2 -no-pie -nostartfiles;./media2
suma = 4294967280 = 0xfffffffff0 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media2.s
-o media2 -no-pie -nostartfiles;./media2
suma = 4294967296 = 0x100000000 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media2.s
-o media2 -no-pie -nostartfiles;./media2
suma = 68719476720 = 0xfffffffff0 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$

```

Código 3

```
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media3.s -o media3 -no-pie -nostartfiles;./media3
suma = -16 = 0xffffffffffffff0 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media3.s -o media3 -no-pie -nostartfiles;./media3
suma = -16 = 0xffffffffffffff0 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media3.s -o media3 -no-pie -nostartfiles;./media3
suma = -1600000000 = 0xfffffffffa0a1f000 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media3.s -o media3 -no-pie -nostartfiles;./media3
suma = -3200000000 = 0xfffffffff4143e000 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media3.s -o media3 -no-pie -nostartfiles;./media3
suma = -4800000000 = 0xfffffffffee1e5d000 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media3.s -o media3 -no-pie -nostartfiles;./media3
suma = -32000000000 = 0xfffffffff88ca6c000 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media3.s -o media3 -no-pie -nostartfiles;./media3
suma = 16 = 0x10 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media3.s -o media3 -no-pie -nostartfiles;./media3
suma = 4294967280 = 0xfffffffff0 hex
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ gcc media3.s -o media3 -no-pie -nostartfiles;./media3
suma = 4294967296 = 0x100000000 hex
```

Código 4

```
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ for i in $(seq 1 19); do gcc -x assembler-with-cpp -D TEST=$i -no-pie -nostartfiles media4.s -o media4; printf "__TEST%02d__%35s\n" $i "" | tr " " "-"; ./media4 ; done
__TEST01__-----
media   =          1  resto   =          8
        = 0x 00000001      = 0x 00000008
__TEST02__-----
media   =         -1  resto   =         -8
        = 0x ffffffff      = 0x ffffffff8
__TEST03__-----
media   = 2147483647  resto   =          0
        = 0x 7fffffff      = 0x 00000000
__TEST04__-----
media   = -2147483648  resto   =          0
        = 0x 80000000      = 0x 00000000
__TEST05__-----
media   =         -1  resto   =          0
        = 0x ffffffff      = 0x 00000000
__TEST06__-----
media   = 2000000000  resto   =          0
        = 0x 77359400      = 0x 00000000
__TEST07__-----
media   = -1294967296  resto   =          0
        = 0x b2d05e00      = 0x 00000000
__TEST08__-----
media   = -2000000000  resto   =          0
        = 0x 88ca6c00      = 0x 00000000
__TEST09__-----
media   = 1294967296  resto   =          0
        = 0x 4d2fa200      = 0x 00000000
__TEST10__-----
media   =          1  resto   =          0
        = 0x 00000001      = 0x 00000000
__TEST11__-----
media   =          1  resto   =          1
        = 0x 00000001      = 0x 00000001
__TEST12__-----
media   =          1  resto   =          8
        = 0x 00000001      = 0x 00000008
__TEST13__-----
media   =          1  resto   =         15
        = 0x 00000001      = 0x 0000000f
__TEST14__-----
media   =          2  resto   =          0
        = 0x 00000002      = 0x 00000000
__TEST15__-----
media   =         -1  resto   =          0
        = 0x ffffffff      = 0x 00000000
__TEST16__-----
media   =         -1  resto   =         -1
        = 0x ffffffff      = 0x ffffffff
__TEST17__-----
media   =         -1  resto   =         -8
        = 0x ffffffff      = 0x ffffffff8
__TEST18__-----
media   =         -1  resto   =        -15
        = 0x ffffffff      = 0x ffffffff1
__TEST19__-----
media   =         -2  resto   =          0
        = 0x fffffffe      = 0x 00000000
```

Código 5

```
alvaroogz@asus-ubu:~/Escritorio/EC/practicas/practica2/programas/guion2$ for i in $(seq 1 19); do gcc -x assembler-with-cpp -D TEST=$i  
-no-pie -nostartfiles media4.s -o media4; printf "__TEST%02d__%35s\n" $i "" | tr " " "-"; ./media4; done  
__TEST01__-----  
media      =          1 resto    =          8  
           = 0x 00000001      = 0x 00000008  
__TEST02__-----  
media      =         -1 resto    =         -8  
           = 0x ffffffff      = 0x ffffffff8  
__TEST03__-----  
media      = 2147483647 resto    =          0  
           = 0x 7fffffff      = 0x 00000000  
__TEST04__-----  
media      = -2147483648 resto    =          0  
           = 0x 80000000      = 0x 00000000  
__TEST05__-----  
media      =         -1 resto    =          0  
           = 0x ffffffff      = 0x 00000000  
__TEST06__-----  
media      = 2000000000 resto    =          0  
           = 0x 77359400      = 0x 00000000  
__TEST07__-----  
media      = -1294967296 resto    =          0  
           = 0x b2d05e00      = 0x 00000000  
__TEST08__-----  
media      = -2000000000 resto    =          0  
           = 0x 88ca6c00      = 0x 00000000  
__TEST09__-----  
media      = 1294967296 resto    =          0  
           = 0x 4d2fa200      = 0x 00000000  
__TEST10__-----  
media      =          1 resto    =          0  
           = 0x 00000001      = 0x 00000000  
__TEST11__-----  
media      =          1 resto    =          1  
           = 0x 00000001      = 0x 00000001  
__TEST12__-----  
media      =          1 resto    =          8  
           = 0x 00000001      = 0x 00000008  
__TEST13__-----  
media      =          1 resto    =         15  
           = 0x 00000001      = 0x 0000000f  
__TEST14__-----  
media      =          2 resto    =          0  
           = 0x 00000002      = 0x 00000000  
__TEST15__-----  
media      =         -1 resto    =          0  
           = 0x ffffffff      = 0x 00000000  
__TEST16__-----  
media      =         -1 resto    =         -1  
           = 0x ffffffff      = 0x ffffffff  
__TEST17__-----  
media      =         -1 resto    =         -8  
           = 0x ffffffff      = 0x ffffffff8  
__TEST18__-----  
media      =         -1 resto    =        -15  
           = 0x ffffffff      = 0x ffffffff1  
__TEST19__-----  
media      =         -2 resto    =          0  
           = 0x ffffffff      = 0x 00000000
```